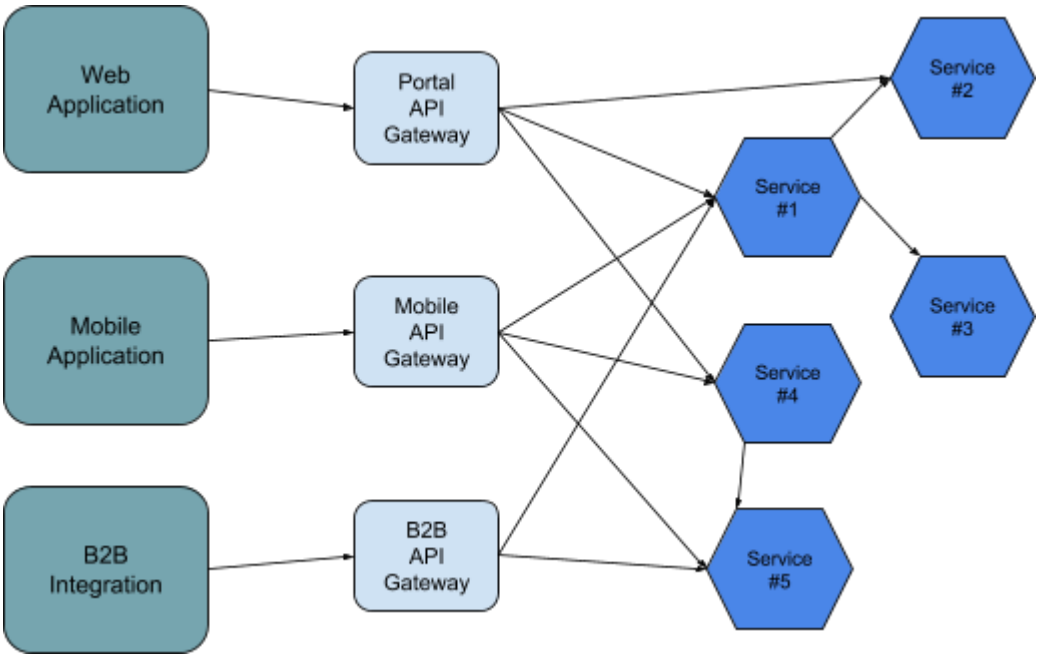
BÀI THỰC HÀNH SỐ 6: CASE STUDY CÀI ĐẶT KIẾN TRÚC MICROSERVICES TRÊN MÔI TRƯỜNG .NET CORE (BÀI TOÁN ĐẠI LÝ BẢO HIỂM)

Phần 4: XÂY DỰNG API GATEWAY VỚI OCELOT

Tiếp theo bài thực hành số 5, sinh viên thực hiện các task tiếp theo như sau:

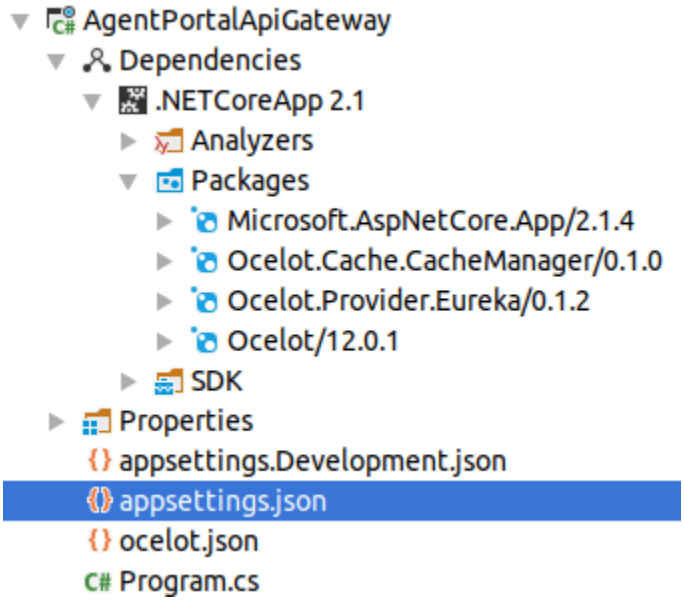


- API Gateway hoạt động như một lớp trung gian che giấu sự phức tạp của hệ thống bên dưới khỏi các ứng dụng khách.
- API Gateway là một microservice khác chạy phía trước các dịch vụ phụ trợ của bạn và chỉ hiển thị các thao tác cần thiết cho ứng dụng khách cụ thể.
- API Gateway có thể làm nhiều hơn là chỉ định tuyến các yêu cầu từ ứng dụng khách đến các dịch vụ backend phù hợp. Tuy nhiên, bạn cần cẩn thận để không đưa vào logic nghiệp vụ và quy trình có thể dẫn đến vấn đề API Gateway quá tham vọng.
- Ngoài việc định tuyến, API Gateway thường chịu trách nhiệm về bảo mật.
-

Task #1: Cài đặt gói: Ocelot

- Thêm hai gói NuGet: Ocelot.Provider.Eureka và Ocelot.Cache.CacheManager.
- Thêm tệp ocelot.json, tệp này sẽ chứa cấu hình cổng Ocelot: *sinh viên kiểm tra, nếu chưa có thì thêm vào.*

Kết quả như trong hình:



- Sửa đổi Program.cs để khởi tạo đúng cách tất cả các dịch vụ cần thiết, bao gồm cả Ocelot: *sinh viên kiểm tra xem code đã đúng với hướng dẫn chưa, nếu chưa thì chỉnh lại cho đúng.*

```
public class Program
{
  public static void Main(string[] args)
  {
     BuildWebHost(args).Run();
  }

  public static IWebHost BuildWebHost(string[] args)
  {
     return WebHost.CreateDefaultBuilder(args)
        .UseUrls("http://localhost:8099")
        .ConfigureAppConfiguration((hostingContext, config) =>
        {
           config
              .SetBasePath(hostingContext.HostingEnvironment.ContentRootPath)
              .AddJsonFile("appsettings.json", true, true)

.AddJsonFile($"appsettings.{hostingContext.HostingEnvironment.EnvironmentName}.jso
n", true,
                    true)
              .AddJsonFile("ocelot.json", false, false)
              .AddEnvironmentVariables();
```

```
            })
            .ConfigureServices(s =>
            {
               s.AddOcelot().AddEureka().AddCacheManager(x =>
    x.WithDictionaryHandle());
            })
            .Configure(a =>
            {
              a.UseOcelot().Wait();
            })
            .Build();
      }
    }
```

- Xem lại cấu hình ở file ***appsettings.json***, nơi thêm cấu hình Eureka : *sinh viên xem code đã có chưa, nếu chưa thì thêm và điều chỉnh code cho đúng theo hướng dẫn dưới đây.*

```
{
  "spring": {
    "application": { "name": "Agent-Portal-Api-Gateway" }
  },
  "eureka": {
    "client": {
      "serviceUrl": "http://localhost:8761/eureka/",
      "shouldRegisterWithEureka": false,
      "validateCertificates": false
    }
  }
}
```

- *GlobalConfiguration* định nghĩa các thiết lập toàn cục cho toàn bộ cổng API: *sinh viên kiểm tra và điều chỉnh code theo hướng dẫn (nếu chưa khớp):*

```
"GlobalConfiguration": {
"RequestIdKey": "OcRequestId",
"AdministrationPath": "/administration",
"UseServiceDiscovery" : true,
"ServiceDiscoveryProvider": { "Type": "Eureka", "Host" : "localhost", "Port" : "8761"}
}
```

- Kích hoạt tính năng khám phá dịch vụ và trỏ đến đúng phiên bản Eureka: định nghĩa các tuyến đường: *sinh viên kiểm tra và điều chỉnh code theo hướng dẫn (nếu chưa khớp):*

```
[serviceHost:port]/api/Products/{code}.
"ReRoutes": [
   {
```

```
    "DownstreamPathTemplate": "/api/Products/{everything}",
    "DownstreamScheme": "http",
    "UpstreamPathTemplate": "/Products/{everything}",
    "ServiceName": "ProductService",
    "UpstreamHttpMethod": [ "Get" ]
  }
]
```

Task #2: ***Bổ sung code cho microservice Service AuthService: 1 thư mục.***

- Copy code từ các file vào project. Trường hợp không add vào được thì tạo thủ công:
  - Tạo các file trùng tên
  - Copy code/nội dung của file vào project
- Lấy code ở file nén đính kèm.
- Trường hợp tên projects hoặc tên các file có khác nhau thì xử lý theo 1 trong 2 phương án:
  - Đổi lại tên cho phù hợp với code mẫu thầy gửi (*khuyên nên dùng phương án này*)
  - Giữ nguyên tên theo project của sinh viên và chú ý đổi tên ở tất cả những nơi mà có được sử dụng.

Task #3: ***Bổ sung code cho microservice Service Chat: 2 thư mục.***

- Copy code từ các file vào project. Trường hợp không add vào được thì tạo thủ công:
  - Tạo các file trùng tên
  - Copy code/nội dung của file vào project
- Lấy code ở file nén đính kèm.
- Trường hợp tên projects hoặc tên các file có khác nhau thì xử lý theo 1 trong 2 phương án:
  - Đổi lại tên cho phù hợp với code mẫu thầy gửi (*khuyên nên dùng phương án này*)
  - Giữ nguyên tên theo project của sinh viên và chú ý đổi tên ở tất cả những nơi mà có được sử dụng.

Task #4: ***Bổ sung code cho microservice Service Dashboard: 3 thư mục.***

- Copy code từ các file vào project. Trường hợp không add vào được thì tạo thủ công:
  - Tạo các file trùng tên
  - Copy code/nội dung của file vào project

- Lấy code ở file nén đính kèm.
- Trường hợp tên projects hoặc tên các file có khác nhau thì xử lý theo 1 trong 2 phương án:
    - Đổi lại tên cho phù hợp với code mẫu thầy gửi (*khuyên nên dùng phương án này*)
    - Giữ nguyên tên theo project của sinh viên và chú ý đổi tên ở tất cả những nơi mà có được sử dụng.

Sản phẩm nộp: link Github chứa solution code.

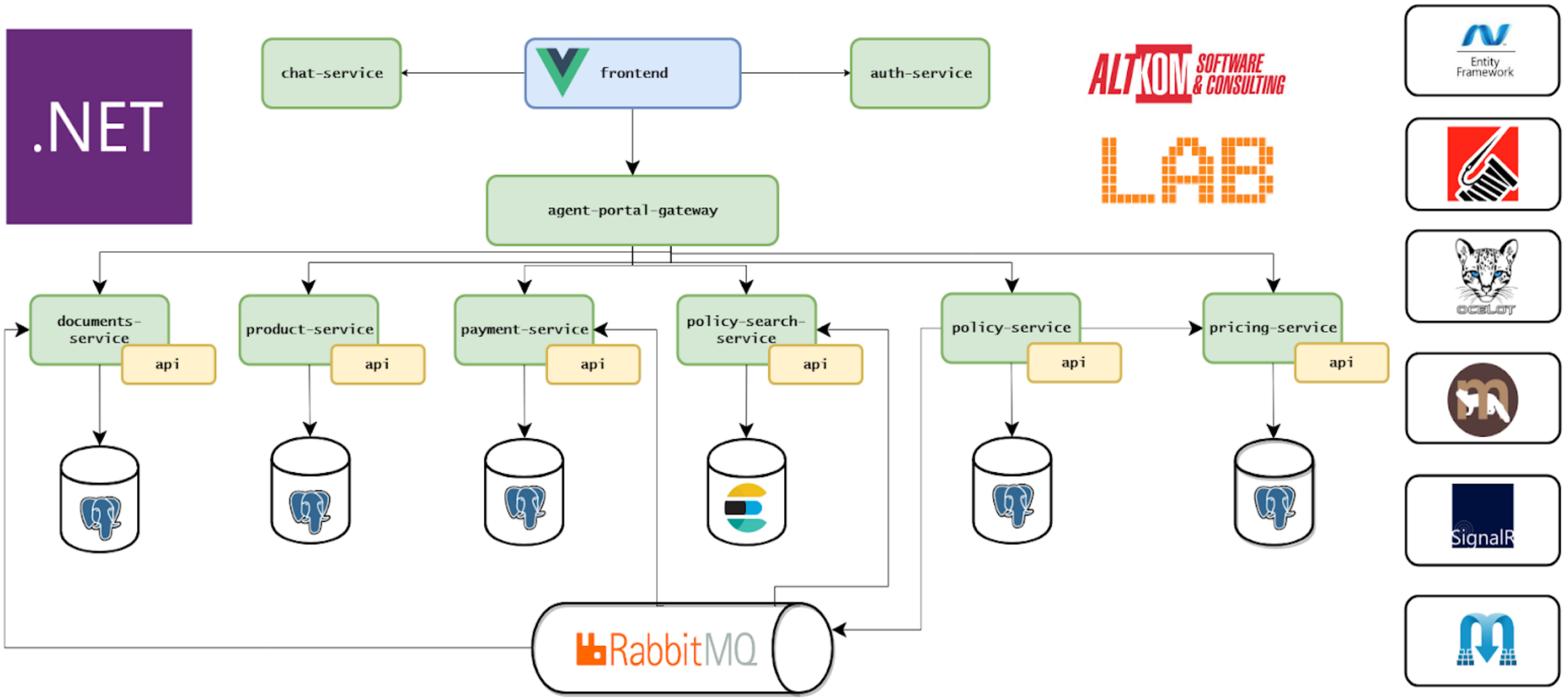
Yêu cầu: Solution không còn lỗi liên quan đến những project vừa thêm solution.

Thời hạn nộp: chậm nhất 18h00 hôm nay (ngày 25/10/2026).

./.

PHỤ LỤC:

KIẾN TRÚC THAM KHẢO THEO YÊU CẦU CỦA BÀI THỰC HÀNH SỐ 3



Hệ thống bao gồm:

1) **Giao diện người dùng (Frontend)** – một ứng dụng trang đơn (Single Page Application) sử dụng VueJS cung cấp cho các đại lý bảo hiểm khả năng lựa chọn sản phẩm phù hợp cho khách hàng, tính toán giá cả, tạo báo giá và hoàn tất quy trình bán hàng bằng cách chuyển đổi báo giá thành hợp đồng bảo hiểm. Ứng dụng này cũng cung cấp các chức năng tìm kiếm và xem thông tin về hợp đồng và báo giá. Giao diện người dùng giao tiếp với các dịch vụ phụ trợ thông qua cổng API.
2) **Cổng API Đại lý (Agent Portal API Gateway)** – là một microservice đặc biệt với mục đích chính là che giấu sự phức tạp của cấu trúc dịch vụ văn phòng phụ trợ khỏi ứng dụng khách. Thông thường, chúng ta tạo một cổng API riêng cho mỗi ứng dụng khách. Trong trường hợp trong tương lai chúng ta thêm ứng dụng di động Xamarin vào hệ thống, chúng ta sẽ cần xây dựng một cổng API riêng cho nó. Cổng API cũng cung cấp rào cản bảo mật và không cho phép các yêu cầu chưa được xác thực được chuyển đến các dịch vụ phụ trợ. Một ứng dụng phổ biến khác của cổng API là tổng hợp nội dung từ nhiều dịch vụ.

3) **Dịch vụ xác thực (Auth Service)** – một dịch vụ chịu trách nhiệm xác thực người dùng. Hệ thống bảo mật của chúng ta sẽ dựa trên mã thông báo JWT. Sau khi người dùng xác thực chính xác, dịch vụ xác thực sẽ cấp một mã thông báo được sử dụng để kiểm tra quyền của người dùng và các sản phẩm có sẵn.
4) **Dịch vụ Chat (Chat Service)** – một dịch vụ sử dụng SignalR để cho phép các nhân viên hỗ trợ trò chuyện với nhau.
5) **Dịch vụ Định giá (Pricing Service)** – một dịch vụ chịu trách nhiệm tính toán giá cho sản phẩm bảo hiểm cụ thể dựa trên các tham số của nó.
6) **Dịch vụ Chính sách (Policy Service)** – dịch vụ chính trong hệ thống của chúng ta. Nó chịu trách nhiệm tạo ra các gói bảo hiểm và chính sách. Nó sử dụng Dịch vụ Định giá thông qua các cuộc gọi HTTP REST để tính toán giá. Sau khi một chính sách được tạo, nó sẽ gửi sự kiện không đồng bộ đến bus sự kiện (RabbitMQ) để các dịch vụ khác có thể phản hồi.
7) **Dịch vụ Tìm kiếm Chính sách (Policy Search Service)** – mục đích duy nhất của dịch vụ này là cung cấp chức năng tìm kiếm. Dịch vụ này đăng ký các sự kiện liên quan đến vòng đời của chính sách và lập chỉ mục chính sách đã cho trong ElasticSearch để cung cấp khả năng tìm kiếm nâng cao.
8) **Dịch vụ Thanh toán (Payment Service)** – dịch vụ này chịu trách nhiệm quản lý các hoạt động tài chính liên quan đến chính sách. Nó đăng ký các sự kiện liên quan đến chính sách, tạo tài khoản chính sách khi chính sách được tạo, đăng ký các khoản thanh toán dự kiến. Nó cũng có một tác vụ nền rất đơn giản để phân tích tệp chứa các khoản thanh toán đến và gán chúng vào tài khoản chính sách phù hợp.
9) **Dịch vụ Sản phẩm (Product Service)** – đây là danh mục sản phẩm. Nó cung cấp thông tin cơ bản về từng sản phẩm bảo hiểm và các thông số có thể tùy chỉnh khi tạo báo giá cho khách hàng.
10) **Dịch vụ Tài liệu (Document Service)** – dịch vụ này sử dụng JS Report để tạo chứng chỉ PDF.

Mỗi microservice được xây dựng xung quanh ngữ cảnh giới hạn (bounded context) theo thuật ngữ DDD. Bạn cũng có thể thấy rằng chúng hợp tác để đạt được mục tiêu kinh doanh chính – bán sản phẩm bảo hiểm cho khách hàng cuối cùng.